Progress report

Plate-specific recognizer

Status as of 17 October 2026, for the 20 October "second approach" milestone

Code: `https://github.com/S1norin/inno-detecting-license-plates`

Contributors

Name	Email
Mikhail Krylov	m.krylov@innopolis.university
Maximilian Mifsud Bonici	m.mifsudbonici@innopolis.university
Andrei Polevoi	a.polevoi@innopolis.university
Evgenii Khovalyg	e.khovalyg@innopolis.university

1 Milestone status

Recognizer trained specifically on license plates. It works and it is much better than the EasyOCR baseline from week 1.

In week 1 the recognizer was weak. The new recognizer reads 75% of characters correctly on clean plate crops, compared to 35% for EasyOCR. On the full pipeline (detector + recognizer) it goes from 32% to 66%. It also reads whole plates correctly: 269 out of 800, where EasyOCR managed 0.

2 Second approach

2.1 What changed

In week 1 we found that EasyOCR does not work well on plate crops. Its exact-plate score was 0 on the test set. So we trained our own recognizer from scratch on CCPD plate crops. A model trained on plates should know what plates look like, which a general OCR model does not.

2.2 Model

The model is a CRNN: convolutional layers that read the image, two bidirectional LSTM layers that track the sequence of characters, and a CTC output that turns the sequence into text.

- Input: one grayscale crop, 48 px tall, up to 256 px wide, aspect ratio kept.
- Character set: the 58 characters that appear in the training plates.
- Size: about 3 million parameters.
- Code: `models/crnn/`.

2.3 Training setup

Setting	Value
Data	3,500 original-scale train crops (700 val crops for checking)
Optimizer / learning rate	Adam, 0.001
Batch size / epochs	32 / 50
Image size	48×256
Seed / determinism	42, deterministic mode
Hardware	CPU only, one thread
Runtime	about 38 minutes for 50 epochs

Table 1: Training setup.

We kept the run fully reproducible: same seed, deterministic mode, one thread. Two independent runs produced byte-identical checkpoints.

2.4 Evaluation protocol

We use the same evaluation harness as week 1, so the numbers are directly comparable. Two protocols:

- **OCR-only** — the recognizer reads clean plate crops. This measures the recognizer alone.
- **End-to-end** — the YOLO detector finds the plate first, then the recognizer reads it. This is the full system.

For each protocol we pick the preprocessing mode and confidence threshold on the validation set only. The test set is then run exactly once with that fixed policy. The test set was never used to choose anything.

3 Results

3.1 Recognition: baseline vs second approach

Protocol	Engine	Coverage	Char. accuracy	Exact plates	Precision	Recall	F1
ocr_only	EasyOCR	0.7300	0.3474	0 / 800	0.0377	0.0924	0.0535
ocr_only	CRNN	1.0000	0.7538	269 / 800	0.5500	1.0000	0.7097
end_to_end	EasyOCR	0.7025	0.3240	0 / 800	0.0267	0.0593	0.0368
end_to_end	CRNN	0.9762	0.6561	133 / 800	0.3624	0.9371	0.5226

Table 2: Test-split recognition results. Character accuracy is the share of plate characters read correctly. “Exact plates” counts plates read fully correct.

The main numbers:

- Clean crops: 75.4% of characters correct, up from 34.7%.
- Full pipeline: 65.6% of characters correct, up from 32.4%.
- Whole plates: 269 out of 800 correct, up from 0.

3.2 Ablations

3.2.1 Preprocessing (E3)

Mode	Coverage	Char. accuracy
none	1.0000	0.7430
gray_contrast	0.9986	0.7264
canny	0.9857	0.4673

Table 3: Validation-split preprocessing ablation (ocr_only).

The trained model works best on the raw crop. Contrast enhancement helps slightly less; Canny edge detection hurts a lot. The selection procedure correctly picked "none" for both protocols.

3.2.2 Size buckets and tilt bins (E4/E5)

Bucket / bin	Char. accuracy (ocr_only)	Char. accuracy (end_to_end)
small / 0–9 deg	0.8635 / 0.7765	0.7564 / 0.6738
medium / 10–19 deg	0.8222 / 0.5130	0.7275 / 0.4340
large / 20+ deg	0.5722 / 0.7738	0.4806 / 0.6754

Table 4: Test-split recognition by plate size (rows 1–3) and tilt bin (rows 1, 2, 4–6; tilt bin 20+ has 570 of 800 samples).

Two things stand out:

- Large plates are read much worse than small ones (57% vs 86%).
- The 10–19 degree tilt bin is the weakest (51%, only 63 samples).

3.2.3 Confidence threshold (E6)

We swept the confidence threshold from 0.50 to 0.99 on validation and picked the threshold that keeps the most plates without losing accuracy. Both protocols selected threshold 0.50 with the "none" preprocessing mode. The test set was then run once with that fixed policy.

4 Observations

- The plate-specific model is the fix we needed. It doubled character accuracy and went from 0 to 269 exact plates on clean crops.
- The full pipeline is still a bit below the clean-crop number (65.6% vs 75.4%). Part of that gap is the detector missing 17 plates, and part is that a detector box is a rougher crop than the ground-truth box.
- Large plates suffer because we squash them into a 48 px tall input. The characters become tiny. A taller input for large plates is the obvious next step.

- The 10–19 degree bin is the weakest tilt group. It is also the smallest, so we should be cautious about over-interpreting it.
- Canny preprocessing destroys most of the signal for a trained model. The selection correctly avoided it.

5 Remaining work

- Fine-tune YOLO11 on our training split and compare it to the pretrained detector. Then re-run end-to-end recognition with the better detector.
- Try a taller input for large plates to improve their accuracy.
- Final submission (10 November): failure-case analysis, runtime and memory numbers, reproducibility check on a clean clone, two-page summary, and demo.

Limitations

- Our split is custom, so the numbers are not comparable to official CCPD benchmark results.
- Tilt bins come from a filename proxy and have very unequal sizes (570 vs 167 vs 63).
- Timing measurements so far cover 100 sampled images, not the full splits.

References

- Dataset format and usage: `data/README.md`
- Detection baseline: `baseline/detection/README.md`
- Recognition baseline: `baseline/ocr/README.md`
- CRNN details and reproduction: `models/crnn/REPORT.md`
- Combined evaluation: `baseline/evaluation/REPORT.md`
- Repository: `https://github.com/S1norin/inno-detecting-license-plates`